Ders 3 Çalışma Özeti

Ders 3 Çalışma Özeti

Genel Konular

- Diyagramların kısa tarihi
  - Algoritmalar uzun süre akış diyagramlarıyla ifade edildi. Kâğıda algoritma çizmek için sembol şablonlu cetveller bile vardı.
  - Zaman içinde başka diyagram teknikleri de ortaya çıktı (Jackson Design, Structured Design gibi). 80'lerin sonu ve 90'larda nesneye yönelik teknikler yaygınlaştı.
  - OMT (Object Modeling Technique), General Electric'te James Rumbaugh ve ekibi tarafından geliştirildi. Kitabın (Object-Oriented Modeling and Design with UML, 2. baskı) kapağındaki küpün üç yüzünde sınıf diyagramı, durum geçiş diyagramı ve ardışıl diyagram vardır. Bu üç yüz, yazılımın yapısal, davranışsal ve işlevsel yönlerinin bir arada ifade edilmesini simgeler.
  - Ders sonrası not: Hoca OMT'yi "Object Management Technique" diye açtı; doğrusu Object Modeling Technique'tir. "Object Management" adı, UML'i standartlaştıran Object Management Group'a (OMG) aittir.
- UML'in ortaya çıkışı (ekrandan, 00:10:00)
  - OMT, değişik teknikleri bir araya toplama çalışmalarının gelişmesine neden oldu. En bilineni Tümleştirilmiş Modelleme Dili UML'dir (Unified Modeling Language).
  - OMG (Object Management Group) tarafından standart hâline getirildi; teknik şartnameler OMG'nin sayfasındadır.
  - IBM tarafından desteklendi: Rational firması, Rational Rose ürünü. Bugün pek çok ticari ve bedava ürün vardır.
  - UML'in üç ana mimarı "Three Amigos": James Rumbaugh, Grady Booch, Ivar Jacobson. UML dışında 3 kitap, 1 el kitabı ve 1 kullanıcı kılavuzu üretmişlerdir.
- Analiz ve tasarım diyagramları
  - UML diyagramları hem analizde hem tasarımda kullanılır. Analiz, çözülen problemle (problem uzayı) ilgilidir; tasarım, problemin çözümünün teknik ayrıntısıyla ilgilidir.
  - Aynı diyagram türü kullanılsa bile analizde kavramsal sınıflar, tasarımda yazılım sınıfları yer alır.
  - UML 2'de 14 diyagram türü vardır. Bir mimariyi ifade etmek için bunların 3-4 tanesi yeterlidir.
- Diyagramların iki ailesi
  - Yapısal diyagramlar statiktir: sistemin her zaman var olan parçalarını ve aralarındaki ilişkileri tanımlar (bir "blueprint" gibi).
  - Davranışsal diyagramlar dinamiktir: sistemin nasıl çalıştığını, sorumluluklarını nasıl yerine getirdiğini gösterir. Her davranış için ayrı bir diyagram gerekir.
- Diyagram kullanmanın faydası ve zararı
  - Fayda: iletişim (ekip, hiç konuşulmayan paydaşlar, yıllar sonra bakımı yapacak kişiler), mevcut yöntemlere destek (ör. tasarım gözden geçirme), araç desteği (CASE araçları hatalı bağlantıları ve yanlış yerdeki sorumlulukları uyarır).
  - Zarar: zaman alır ve sistem değiştikçe bütün diyagram ailesinin güncel tutulması gerekir.
- Yapısal diyagramlar (7 tane)
  - Sınıf diyagramı: en popüler diyagramdır ("statik model", "sınıf yapı diyagramı"). Sınıfları (ad, özellikler, işlemler) ve ilişkileri gösterir. Üç ilişki: bağımlılık (kesikli ok, "X, Y'yi kullanır"), ilişki (association; içerme varsa ucunda eşkenar dörtgenle aggregation) ve genelleştirme ("X bir Y tipindedir"). Örnek: film, fiyat (çocuk, yeni çıkan,

normal fiyat alt sınıfları), kiralama, müşteri.
  - Nesne diyagramı: belirli bir T anında çalışan nesnelerin fotoğrafı (ör. `c` Company'den, `d1`, `d3` Department'tan türemiş, `p` Person, ContactInformation).
  - Kompozit yapı diyagramı (ekrandan, 00:20:00): sınıfın iç yapısını ve arayüzlerini gösterir. Sağladığı arayüz (provides interface): sınıf dünyaya hangi yetenekleri sunuyor. Gerektirdiği arayüz (requires interface): sınıf dünyadan ne bekliyor. Sınıflar bu arayüzler üzerinden priz gibi birbirine bağlanır; port ve part öğeleri de vardır.
  - Bileşen (component) diyagramı: mimariyi ifade eden diyagramların temelidir. UML referans el kitabına göre bileşen, sistemin fiziksel ve değiştirilebilir bir parçasıdır; gerçekleştirmeyi kapsar, bir arayüz kümesine uyar ve onu gerçekleştirir. Bileşenler arası ilişki, bir bileşenin diğerinin servislerini kullanmasıdır. Örnek: Ticket Database (status ve purchase arayüzleri), Ticket Seller (Subscription Sales, Individual Sales, Group Sales); grup satışı yalnız gişe görevlisi (Clerk) üzerinden yapılır, kiosktan yapılmaz.
  - Barındırma (deployment) diyagramı: bileşenlerin hangi işlem birimlerinde (düğüm, node) çalıştığını ve düğümler arası haberleşmeyi (kenarlar) gösterir. Örnek: istemcideki Planner bileşeni, sunucudaki Scheduler bileşeninin Reservation arayüzüyle doğrudan kanal (Direct Channel) üzerinden haberleşir; Meeting DB sunucudadır.
  - Paket diyagramı: genel amaçlı düzenleme mekanizmasıdır. Her paketin kendi ad uzayı vardır; aynı adlı öğeler farklı paketlerde çakışmaz. Paket içindeki parçalar arasında bağımlılık varsa paketler arasında da gösterilir. Paketler iç içe olabilir (klasör ayracı simgesi). UML 2.0'da içerme ve import gösterimleri vardır.
  - Profil diyagramı: UML'i UML ile tanımlamaya (meta model) dayanır. Meta sınıflar ve stereotiplerle UML genişletilir; kendi UML profilinizi tanımlarsınız.
- Yapısal diyagramların özeti (ekrandan, 00:30:00)
  - Sınıf: bileşenler ve yapısal özellikleri. Kompozit yapı: iç yapı ve olası etkileşimler. Bileşen: fiziki yazılım bileşenlerinin organizasyonu. Barındırma: fiziki sistem kaynakları ve donanıma dağıtımları. Nesne: belirli bir zamandaki statik yapı. Paket: mantıksal gruplama ve bağımlılıklar. Profil: UML meta modeline yapılan eklentiler.
  - Diyagramlarda fazla sayıda örtüşme vardır; bu normaldir.
  - Mimarinin statik yapısı için: sınıf, bileşen, barındırma ve paket diyagramları.
- Davranışsal diyagramlar (7 tane)
  - Kullanım senaryosu (use case) diyagramı: OMT analizde veri akış diyagramlarını önerirken OMG, Jacobson'ın kullanım senaryolarını önerir. Diyagram, use case modellemesinin yalnız basit bir resmidir; asıl iş senaryoları bulmak ve yazmaktır. Çöp adamlar dış aktörlerdir (insan, başka sistem, cihaz); elipsler senaryolardır. Diyagram bütün senaryoların kümesini tek resimde gösterir: her elips için ayrı bir senaryo yazılır.
  - Senaryo ilişkileri: düz çizgi katılım demektir. `<<extends>>`: zorunlu olmayan, duruma göre senaryoyu genişleten parça ("bir alana iki bedava"). `<<include>>`/`<<uses>>`: alt program çağrısı gibi, pek çok senaryoda ortak davranış (ör. kimlik doğrulama).
  - Bağlam (context) diyagramı: UML'in parçası değildir. Seviye 0 veri akış diyagramına eşittir: tek işlem birimi sistemin kendisidir; sistemin dış paydaşlardan aldıkları ve onlara verdikleri gösterilir.
  - Ardışıl (sequence) diyagram (ekrandan, 00:42:30): bir senaryoyu açıklar. Her kolonda tekil bir katılımcı (genellikle nesne), zaman aşağı doğru akar, yatay çizgiler nesneler arası mesajlardır, kalın kutular metot gövdeleridir. Telefon sanayisindeki mesaj sırası kartlarından evrimleşmiştir.
  - İletişim (communication) diyagramı: ardışıl diyagramla birebir aynı içeriktedir; mesajlar 1, 2, 2.1, 2.2, 3 gibi ondalık numaralarla sıralanır, yer tasarrufu sağlar.
  - Aktivite diyagramı: Petri ağlarından türemiştir. İş akışını, paralel işlemleri ve senkronizasyonu gösterir; ardışıl ya da iletişim diyagramıyla gösterilemeyen eşzamanlılık burada gösterilir. Siyah bar senkronizasyon noktasıdır (çatallanma ve birleşme).
  - Genel etkileşim (interaction overview) diyagramı: aktivite diyagramının üst seviye hâlidir; düğümleri ardışıl, iletişim, zamanlama ya da başka genel etkileşim diyagramlarıdır.
  - Zamanlama (timing) diyagramı: dijital çip tasarımından gelir; zaman soldan sağa (ör. 0–120 ms) akar, oklar senkronizasyon noktalarını gösterir.
  - Durum (state) diyagramı (ekrandan, 00:50:00): davranışsal diyagramların en güçlü ve karmaşık olanıdır. Sonlu durum makinelerini birleştirme (aggregation), eşzamanlılık, geçmiş ve olayların tüme yayılımı ile genişletir. Örnek: Idle ve Maintenance

durumları; Maintenance içinde eşzamanlı Testing ve Commanding alt durumları.

- OCL (Object Constraint Language, nesne kısıt dili) (ekrandan, 00:54:30)
  - Diyagramla ifade edilemeyen kısıtlar için UML'in metin uzantısıdır; genellikle sınıflarda ve durum geçiş diyagramlarında kullanılır.
  - Örnek: `context Account::deposit(Real : amount)`, `pre: amount > 0`, `post: balance = balance@pre + amount`. `pre` işlemin çalışması için gereken koşulu, `post` işlem bittiğinde garanti edilen durumu tanımlar.
  - Ders sonrası not: Slayttaki parametre sırası `Real : amount` biçimindedir. OCL ve UML'de önce ad, sonra tip yazılır: `context Account::deposit(amount : Real)`.
- UML meta modeli
  - Meta modelde Class'ın association, attribute ve operation'ları vardır; Class bir Classifier'dır; operation'ların parametreleri vardır. Meta model profil diyagramıyla değiştirilebilir.
- Nesneye yönelik analiz
  - Tasarlamadan önce problemi anlamak gerekir; anlaşılan problemin gösterildiği aşama analizdir.
  - Doğal dille tanımlanan gerçek dünya nesneleri yazılım uzayına taşınır ve sınıf diyagramıyla ifade edilir. Alternatif, işlevlere odaklanan fonksiyonel (yapısal) çözümlemedir.
  - Neden fonksiyon yerine nesne (ekrandan, 01:20:00): yazılımların uzun yaşam ömürleri vardır ve evrimsel değişikliklere uğrarlar. Bakım sırasında fonksiyonlar nesnelerden çok daha sık değişir. Bankacılıkta faiz hesabı, işlem ücreti, vergi gibi fonksiyonlar değişir; hesap, müşteri, işlem gibi nesneler değişmez. Nesneler daha kararlıdır ve daha sürdürülebilir tasarım sağlar.
  - Yapısal analizde işlevsellik ön plandadır, veri ikinci plandadır. Nesneye yönelik analizde önce kavramlar, özellikleri ve veri tipleri modellenir, davranışlar sonra eklenir.
- Dilbilgisinden modele
  - Gereksinim metni (SRS) ve senaryolar alınır. İsimler sınıfları, sıfatlar özellikleri, eylem bildiren fiiller işlemleri, durum bildiren fiiller ilişkileri verir.
  - Yöntem 1980'lerde Abbott ve arkadaşlarıyla ortaya çıkmıştır.
- Örnek: bir ağacın yapraklarını saymak (ekrandan, 01:24:45)
  - Ağaç: döngü içermeyen çizge; her düğümün sıfır ya da daha fazla alt düğümü ve en fazla bir üst düğümü olabilir.
  - Yöntem: sayılmamış ağaç parçalarını boş yığına koy; yaprak sayacını sıfırla; yığın boş olmadığı sürece ağaç alıp incele; ağaç tek yapraksa sayacı artır ve ağacı at; iki alt ağacı varsa sol ve sağı ayırıp ikisini de yığına ekle; yığın boşalınca yaprak sayısını göster.
  - Aday sınıflar: Counter, Leaf, Tree, Pile (yığın; Stack yerine özel bir ad). "Parça" yerine "alt ağaç" daha iyi bir addır. "Sıfır" bir sınıf değil, sayacın ilk değeridir; silinmemelidir.
  - Özellikler (ekrandan, 01:33:00): Counter `count`; Tree `left subtree`, `right subtree`, `leaf/leaves`, `part`; Pile `empty`.
  - Son sınıf modeli (ekrandan, 01:40:00): Counter (`-count`; `-increment`, `-display`), Leaf, Tree (`-left subtree`, `-right subtree`, `-leaf/leaves`, `-part`; `-split`, `-throw away`), Pile (`-empty`; `-put a Tree onto`, `-get a Tree from`, `-take a tree off`). Counter ile Leaf arasında `counts` ilişkisi; Leaf, Tree'nin alt sınıfı (genelleştirme); Pile ağaçlardan oluşur (aggregation); Tree de ağaçlardan oluşur (kendine aggregation).
- UML sınıf modeli ayrıntıları
  - Sınıf, benzer örneklerin şablon tanımıdır. Alan nesneleri, roller, olaylar ve kendi özelliği olan etkileşimler sınıf olabilir.
  - Ad zorunludur. Eğik yazılmış ya da `<<abstract>>` stereotipiyle verilmiş ad soyut sınıf demektir; soyut sınıftan örnek türemez, alt sınıfların ortak özelliklerini tanımlar.
  - Stereotip (klişe) ile temel modelleme dili genişletilir.
  - Özellikler alanı (ekrandan, 01:47:00): ad, tip ve erişim türü. `+` genel, `-` özel, `#` korumalı (yalnız alt sınıflar), `~` paket içinde görünür. Çoğullama belirtilebilir; tipler tanımlanmalıdır; ilk değer ya da türetilmiş (hesaplanan) özellik belirtilebilir; `{frozen}` değerin değiştirilemeyeceğini gösterir.
  - İşlemler alanı (ekrandan, 01:50:00): anlamlı ad, görünürlük, dönüş tipi, parametre tipleri, ilk değer ve içeri/dışarı tanımları; `{query}`, `{concurrency}`, `{abstract}`; sınıf kapsamındaki işlem tekil nesnelere ait değildir (ör. bu sınıftan kaç nesne türetildi?).
  - Örnek sınıflar (ekrandan, 01:50:15): Rectangle (`p1:Point`, `p2:Point`; `<<construc-`

tor>> Rectangle(p1:Point, p2:Point); <<query>> area(): Real, aspect(): Real; <<update>> move(delta: Point), scale(ratio: Real)). Reservation: operations guarantee(), cancel(), change(newDate: Date); responsibilities bill no-shows, match to available rooms; exceptions invalid credit card.
  - İleri sınıf öğeleri: arayüzler, parametrik sınıflar (Java generic, C++ template), iç içe sınıflar, kompozit nesneler.
  - İleri ilişkiler: association (insan araç kullanır), generalization (araba bir araçtır), dependency (emisyon kanunları değişirse arabalar uyarlanmalıdır; gizli bağımlılık da gösterilmelidir). Poligon en az 3 sıralı noktadan oluşur ve noktalar bu ilişkide vertex rolündedir.
  - İlişki sınıfları (ekrandan, 01:55:40): Company ile Person arasındaki Job ilişkisinde maaş (salary) kişinin değil, çalışma ilişkisinin özelliğidir. İlişki sınıfı, ilişki çizgisine kesikli çizgiyle bağlanır. Job'un kendine Manages ilişkisi vardır (boss 0..1, worker *): patron yönetir işçiyi.
  - Birleşme (aggregation) ve bütünleşme (composition): birleşme içi boş eşkenar dörtgendir ve yaşam süresi hakkında bilgi vermez. Bütünleşmede parçaların yaşam süresi bütüne bağlıdır ve geçişkendir. Ev ile oda bütünleşmedir; oda ile masa birleşmedir.
  - Niteleyiciler (qualifiers) (ekrandan, 02:01:20): sınıf dörtgeninin kenarındaki küçük dörtgenler; birincil anahtar gibi nesneye erişim sağlar. Bank account# üzerinden Person'a; Chessboard rank: Integer, file: Integer üzerinden Square'e (burada bütünleşme de vardır).
  - Genelleme (ekrandan, 02:04:00): üst sınıf üçgen tarafındadır. Alt sınıfın örnekleri üst sınıfın da örneğidir, ama bu kalıtımla birebir aynı kavram değildir. Bir nesne birden fazla alt sınıfa ait olabiliyorsa {overlapping}, tek birine aitse {disjoint}. Alt sınıflar bütün nesneleri kapsıyorsa {complete}, kapsamıyorsa {incomplete}. Örnek: Vehicle; power (WindPowered, MotorPowered) ve venue (Land, Water); Sailboat hem WindPowered hem Water, Truck hem MotorPowered hem Land.
- UML diyagramlarının hiyerarşisi
  - Bütün diyagramlar UML'de Diagram sınıfının altında genelleştirme ilişkisiyle yer alır: 7 yapısal ve 7 davranışsal, toplam 14 diyagram.

Hocanın Özellikle Vurguladığı Kısımlar

- UML yalnız sınıf diyagramı değildir
  - Pek çok kişi için "UML = sınıf diyagramı"dır; hocaya göre bu yanlış bir kullanımdır.
- Bütün diyagramları kullanmak zorunlu değil
  - Probleme en iyi katkıyı sağlayan diyagramlar seçilmelidir. Mimarinin statik yapısı için sınıf, bileşen, barındırma ve paket diyagramları; davranış için çoğunlukla ardışıl diyagramlar kullanılır.
- Use case diyagramı bir hikâye resmi değildir
  - Sık yapılan hata, diyagramı aktör-baloncuk hikâyesi gibi çizmektir. Diyagram bütün senaryoların kümesidir; asıl iş senaryo metnini yazmaktır. Sekiz senaryo varsa sekiz elips beklenir.
- Gizli bağımlılıklar
  - X, Y üzerinden Z'ye erişip Z'nin bir şeyini kullanıyorsa X aslında Z'ye bağımlıdır; bu bağımlılık diyagramda açıkça gösterilmelidir. Emisyon kanunu örneğindeki gibi görünmeyen bağımlılıklar da gösterilmelidir.
- Genelleştirme kalıtımla birebir aynı değildir
  - UML'de modelleme yapılır, gerçekleştirme değil. Çoklu üst sınıf, discriminator ve kısıtlar gibi programlama dilindeki extends'ten fazlası ifade edilebilir.
- Analiz modelinde tek doğru yoktur
  - Hocanın modelinde Leaf ayrı bir alt sınıftır; Leaf'i hiç koymayıp Counter'ı doğrudan Tree'ye bağlamak da bir çözümdür. Tasarım öğretilmez, yaparak öğrenilir.
  - Model artımlı olarak gelişir ve müşteriye sık sık geri dönülür. Problem uzayı yeterince modellenince bir yerde durmak gerekir.
- Sorumlulukları doğru yere vermek
  - Push/pop yığının, artırma sayacın, kendini bölme ve atma ağacın sorumluluğu olmalıdır. Kararlar iyi uyum ve az bağımlılık ilkelerine göre verilmelidir.
- Ana sistem sınıfı

  - Her sistemde `main`'in bulunduğu bir sistem sınıfı vardır (ör. "ağaç yaprak sayma sistemi"). Analiz diyagramlarında genellikle gösterilmez, gerçekleştirmede mutlaka gerekir.
- Özellik tipleri basit olmalı
  - Karmaşık bir tip varsa orada aslında bir aggregation ya da composition vardır; o tip ayrı bir sınıf olarak modellenmelidir.
- Vejetaryen mi omnivor mu?
  - Hocaya göre kafa karıştırıcı bir örnek: omnivor daha geniş gibi görünse de vejetaryen üst sınıftır, çünkü omnivor ona davranış ekleyerek onu özelleştirir.

Kısa Tekrar Notları

- OMT: Rumbaugh, General Electric. UML: OMG standardı, Rational Rose, Three Amigos (Rumbaugh, Booch, Jacobson).
- Analiz = problem uzayı, kavramsal sınıflar. Tasarım = çözüm, yazılım sınıfları.
- UML 2: 14 diyagram = 7 yapısal (statik) + 7 davranışsal (dinamik).
- Yapısal: sınıf, nesne, kompozit yapı, bileşen, barındırma, paket, profil.
- Davranışsal: use case, aktivite, ardışıl, iletişim, genel etkileşim, zamanlama, durum. Bağlam diyagramı UML'de değildir (Seviye 0 DFD).
- Diyagram faydası: iletişim, yönteme destek, araç desteği. Zararı: zaman ve güncel tutma maliyeti.
- Sınıf diyagramı ilişkileri: bağımlılık (kesikli ok), ilişki (düz çizgi, içerme varsa aggregation), genelleştirme.
- Use case: `<<extends>>` isteğe bağlı genişletme, `<<include>>`/`<<uses>>` ortak alt senaryo.
- Ardışıl ve iletişim diyagramları aynı içerik; iletişimde 1, 2, 2.1, 2.2, 3 numaralandırma.
- Aktivite: senkronizasyon barları ile paralel akış. Durum diyagramı: en güçlü davranışsal diyagram.
- OCL: `pre: amount > 0`, `post: balance = balance@pre + amount`.
- Nesneler fonksiyonlardan kararlıdır; bakımda fonksiyonlar daha sık değişir.
- İsim → sınıf, sıfat → özellik, eylem fiili → işlem, durum fiili → ilişki.
- Ağaç yaprağı sayma modeli: Counter, Leaf, Tree, Pile.
- Görünürlük: + genel, - özel, # korumalı, ~ paket.
- `{frozen}`, `{query}`, `{concurrency}`, `{abstract}`; sınıf kapsamlı işlem.
- İlişki sınıfı: maaş, Job ilişkisinin özelliğidir.
- Ev–oda bütünleşme (composition), oda–masa birleşme (aggregation).
- Niteleyici: birincil anahtar gibi (Bank `account#`, Chessboard `rank`, `file`).
- `{overlapping}`/`{disjoint}`, `{complete}`/`{incomplete}`. Atlet: overlapping + incomplete. Kitap: disjoint + incomplete.

Detaylı Açıklamalar

Dersin ilk saati, bir yazılımın mimarisinin ve tasarımının hangi dille ve hangi diyagramlarla ifade edileceği sorusuna ayrılır. Hoca önce diyagramların tarihine kısaca değinir: algoritmalar akış diyagramlarıyla çizilirdi, sonra pek çok farklı teknik ortaya çıktı. Nesneye yönelik tekniklerin yaygınlaşmasıyla General Electric'te Rumbaugh'un ekibi OMT'yi geliştirdi. OMT'nin gücü, yapısal, davranışsal ve işlevsel üç bakış açısını ortak bir diyagram ailesinde birleştirmesiydi. Bu çalışma UML'e dönüştü, OMG tarafından standartlaştırıldı ve IBM'in Rational Rose ürünüyle yaygınlaştı.

UML diyagramları analizde ve tasarımda kullanılır, ama amaçları farklıdır. Analiz diyagramları problem uzayını modeller, gerçek dünyaya yakındır ve kavramsal sınıflar içerir. Tasarım diyagramları ise problemin nasıl çözüleceğinin teknik ayrıntılarını gösterir ve yazılım sınıfları içerir. UML 2'deki 14 diyagram yapısal ve davranışsal diye ikiye ayrılır. Hoca hepsini tanıtır, ama bir mimariyi ifade etmek için 3-4 tanesinin yeterli olduğunu söyler.

Yapısal diyagramlar sistemin her zaman var olan parçalarını gösterir. Sınıf diyagramı bunların en bilinenidir. Sınıf ve nesne diyagramı arasındaki fark, sınıf diyagramının şablonları, nesne diyagramının ise çalışan sistemin belirli bir andaki fotoğrafını göstermesidir. Kompozit yapı diyagramı sınıfın sağladığı ve gerektirdiği arayüzleri öne çıkarır ve parçaların priz gibi birbirine takılmasını modeller. Bileşen diyagramı mimarinin asıl diyagramıdır: kütüphaneler ya

da hizmet veren parçalar, sundukları ve ihtiyaç duydukları arayüzlerle gösterilir. Bilet satış örneğinde grup satışı arayüzünün yalnız gişe görevlisine bağlanması, kiosktan grup satışı yapılamayacağını modeller. Barındırma diyagramı bu bileşenlerin hangi makinelerde çalıştığını ve makinelerin nasıl haberleştiğini gösterir; tek bakışta istemci ve sunucu katmanları ve aralarındaki bağlantılar görülür. Paket diyagramı mantıksal gruplamayı ve ad uzaylarını, profil diyagramı ise UML'in kendisini genişletmeyi sağlar.

Davranışsal diyagramların başında use case diyagramı gelir. Hoca, use case modellemesinde asıl işin senaryoları bulmak ve yazmak olduğunu, diyagramın bunların yalnız kuş bakışı bir resmi olduğunu vurgular. Senaryo, kullanıcının gözünden sistemin bir etkileşimi baştan sona nasıl karşıladığını anlatan hikâyedir. Hoca örnek olarak Ayşe'nin Amazon'dan ürün almasını verir (ekrandan, 00:40:00): Ayşe ürün almak ister, siteye girer, aradığı ürünü bulana kadar gezer, ürünü sepete ekler, satın alma sayfasına geçer, fatura bilgilerini verir ve ödemeyi tamamlar. Her cümle sistem için bir sorumluluk doğurur. Senaryo tablo hâlinde de yazılabilir; o zaman aktör, eylem ve nesne(ler) için üç kolon gerekir (ör. Ayşe, ekler, Kitap, Sepet).

Ardışıl diyagram bir senaryodaki tek bir sorumluluğun zaman içinde nasıl yerine getirildiğini gösterir. Slayttaki örnekte Client bir Transaction nesnesi yaratır, `setActions` çağırır; Transaction, ODBCProxy'ye `setValues` çağrıları yapar, commit bildirilir ve Client Transaction'ı yok eder. Aynı içerik iletişim diyagramında numaralı mesajlarla gösterilir. Paralel işler ve senkronizasyon gerekiyorsa aktivite diyagramı kullanılır: siyah barda akış çatallanır, işler farklı sürelerde bitse de sonraki barda beklenir ve akış birleşir. Belirli zamanlamalar gerekiyorsa zamanlama diyagramı, sistemin durumlarını ve durumlar arası geçişleri modellemek için durum diyagramı kullanılır.

Bazı şeyler diyagramla ifade edilemez. Bunlar için OCL kullanılır. Hesap sınıfının para yatırma işleminde miktarın sıfırdan büyük olması ön koşul, bakiyenin eski bakiye artı miktar olması son koşuldur. Hoca ayrıca 18 yaşından küçükler için bir davranışın çalışmaması gibi kısıtların da diyagramın yanına not düşmek yerine OCL ile formal olarak yazılması gerektiğini söyler.

İkinci saat nesneye yönelik analize ayrılır. Hoca önce neden fonksiyonlar yerine nesnelerin seçildiğini açıklar: yazılımlar uzun yaşar, değişir ve bu sırada fonksiyonlar nesnelerden çok daha sık değişir. Bankacılık örneğinde vergi oranı ya da işlem ücreti değişse de hesap ve müşteri kavramları aynı kalır. Analizde gereksinim metnindeki isimler, sıfatlar ve fiiller işaretlenir ve modele dönüştürülür. Hoca bunun kolay göründüğünü, ama her adımda düşünerek karar vermek gerektiğini ağaç yaprağı sayma örneğiyle gösterir. "Ağaç", "alt ağaç", "sol ağaç", "sağ ağaç" aynı kavrama işaret eder; "parça" yerine "alt ağaç" daha iyi bir addır; "yaprak" aslında özel bir ağaçtır; "sıfır" sınıf değil, sayacın ilk değeridir. Analist metinde olmayan bir kavramı da (ör. düğüm) ekleyebilir. Sonunda dört sınıf kalır: Counter, Leaf, Tree ve Pile. Sorumluluklar da düşünülerek dağıtılır: yığına koyma ve çekme Pile'ın, artırma Counter'ın, kendini bölme ve atma Tree'nin işidir.

Dersin sonunda UML sınıf modelinin ayrıntılarına geçilir: görünürlük işaretleri, çoğullamalar, ilk değerler, türetilmiş özellikler, stereotipler ve özel tanımlar. İlişkiler de ayrıntılandırılır. Bir ilişkinin kendi özellikleri varsa (maaş gibi) ilişki sınıfı olarak modellenir. Birleşme ile bütünleşme farkı yaşam süresidir: ev yıkılınca odaları da yıkılır (bütünleşme), ama masa başka yerde kullanılmaya devam eder (birleşme). Niteleyiciler bir sınıfın nesnelerine bir anahtar üzerinden erişimi gösterir. Genellemede `{overlapping}`, `{disjoint}`, `{complete}` ve `{incomplete}` kısıtları kullanılır. Atletler hem voleybol hem futbol oynayabildiği ve hentbol oyuncuları gibi listede olmayanlar bulunduğu için overlapping ve incomplete'tir. Bir kitap aynı anda hem kâğıt hem bez ciltli olamayacağı için disjoint, e-kitap gibi başka türler olduğu için de incomplete'tir. Ders, bütün 14 diyagramın UML'de Diagram sınıfı altında bir genelleştirme hiyerarşisi oluşturduğu hatırlatılarak kapanır.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.